

# Reviewer Pack — Noncommutative $L_p$ Geometry of Tensor Product Matrices vs Co...

Entry d6766be9a421 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 01:31:32 UTC

## Noncommutative $L_p$ Geometry of Tensor Product Matrices vs Communication Complexity for Disjointness

Entry ID: d6766be9a421 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 01:31:32 UTC

### 1. Conjecture

**Field A** (mathematical branch): noncommutative  $L^p$  geometry **Field B** (complexity object): communication complexity (disjointness)

#### Statement:

[‘For any given disjointness instance  $I$  with  $n$  variables, the noncommutative  $L^2$ -norm of the tensor product matrix  $A_I * B_I$  is at least  $\Omega(n^{1/2})$  where  $A_I$  and  $B_I$  are matrices encoding  $I$  in a noncommutative setting.’, ‘For all instances  $I$  with  $n \leq 40$  and constant  $k$ , if the communication complexity  $C_{\text{DISJ}}(I)$  for disjointness is less than or equal to  $kn^{(-1/2)}$ , then the  $L^2$ -norm of  $A_I * B_I$  is less than  $\epsilon$ , for some small constant  $\epsilon > 0$ .’]

#### Rationale (proposer’s reasoning):

[‘The noncommutative  $L^p$  geometry provides a framework that might capture the geometric properties of matrices encoding communication complexity problems. The tensor product matrix construction could be a useful tool to encode instances in this context.’, ‘Previous work on communication complexity often focuses on commutative structures, but the use of noncommutative  $L^p$  geometry may reveal new insights into the fundamental nature of these problems.’]

**Taxonomy category:** COMM\_DISJ (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** dabfb4c9308dec24

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The noncommutative  $L^2$ -norm of the tensor product matrix  $A_I * B_I$  is at least  $\Omega(n^{1/2})$  for disjointness instance  $I$  with  $n$  variables, and less than  $\epsilon$  if communication complexity  $C_{\text{DISJ}}(I) \leq kn^{(-1/2)}$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - intitle:noncommutative  $L^p$  geometry AND communication complexity AND disjointness - arxiv:math-ph OR arxiv:quant-ph AND abstract:tensor product matrices AND communication complexity - title: $L^2$ -norm tensor product matrices AND communication complexity for disjointness

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=0, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_instance(n):
        instance = [random.choice([0, 1]) for _ in range(n)]
        return instance

    def encode_in_noncommutative_matrices(instance):
        n = len(instance)
        A = [[0] * n for _ in range(n)]
        B = [[0] * n for _ in range(n)]

        for i in range(n):
            for j in range(n):
                if instance[i] == 1 and instance[j] == 1:
                    A[i][j] = 1
                    B[i][j] = 1

        return A, B

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]

        for i in range(n):
            for j in range(n):
```

```

        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]

    return C

def l2_norm(matrix):
    n = len(matrix)
    norm = 0.0

    for i in range(n):
        for j in range(n):
            norm += matrix[i][j] ** 2

    return math.sqrt(norm)

n = random.randint(5, 40)
instance = generate_disjointness_instance(n)
A, B = encode_in_noncommutative_matrices(instance)
C = matrix_multiplication(A, B)
norm = l2_norm(C)

lower_bound = math.sqrt(n)
if norm < lower_bound:
    return {
        "metric_name": "Noncommutative L^2-norm",
        "metric_value": norm,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Lower bound not met: {norm} < {lower_bound}"
    }

return {
    "metric_name": "Noncommutative L^2-norm",
    "metric_value": norm,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```

print(f"RESULT: FALSIFIED counterexample=\"Lower bound not met\" first_failing_seed={first
else:
print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

es_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 196.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 225.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 49.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 36.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 25.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 121.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 49.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 16.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 196.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 64.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 484.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 121.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'Noncommutative L^2-norm', 'metric_value': 100.0, 'instances_tested': 1, 'con
RESULT: SUPPORTED mean=122.4 std=116.697215048175 support_fraction=1.0

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

critic LLM failed:

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Safety rail: critic\_challenge | original judge: The test results show that for all tested instances, the noncommutative  $L^2$ -norm of the tensor product matrix meets the conjectured lower bound of  $\Omega(n)$  | next: Further investigation into the relationship between the noncommutative  $L^2$ -norm and communication complexity for larger instance sizes, as well as exploring the implications of these findings in the context of quantum information theory.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11524        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6549         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4791         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8602         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16199        |

| #  | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|----------|---------------|------------------|-----------|-----|--------------|
| 6  | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10858        |
| 7  | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9237         |
| 8  | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9244         |
| 9  | critic   | ?             | ?                | 0         | 0   | 15136        |
| 10 | judge    | ollama_remote | glm4:latest      | 0         | 0   | 9082         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101222 ms total latency. Provider mix: {'ollama\_remote': 9, '?': 1}

(full prompt+response transcripts available in *research/audit/d6766be9a421.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/d6766be9a421.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/d6766be9a421.tar.gz* (if generated)